information

MDPI

*Article*

# Efficient Software Vulnerability Detection Using Edge-Conditioned Graph Neural Networks on Heterogeneous Code Property Graphs

**Ahmed M. Elalfy [1], Gamal A. Ebrahim [1,*] and Marvy Badr Monir Mansour [2]**

[1] Computer and Systems Engineering Department, Faculty of Engineering, Ain Shams University, Cairo 11517, Egypt; 2200027@eng.asu.edu.eg
[2] Electrical Engineering Department, Faculty of Engineering, The British University in Egypt (BUE), Cairo 11837, Egypt; marvy.badr@bue.edu.eg
* Correspondence: gamal.ebrahim@eng.asu.edu.eg

**Abstract**

Software vulnerabilities are a primary cause of security breaches, and their automated detection at scale has therefore become a pressing concern for both industry and academia. Most Graph Neural Network (GNN) approaches to vulnerability detection treat code graphs as homogeneous structures, and the semantic distinctions between Abstract Syntax Tree (AST) edges, Control-Flow Graph (CFG) edges, and data-flow dependency edges are consequently discarded. The main objective of this study is to determine whether explicitly conditioning message passing on edge type yields accurate yet lightweight detection. To this end, an edge-conditioned GNN named FastVulnGNN is proposed, in which the message-passing computation is conditioned on edge-type information drawn from Code Property Graphs (CPGs). FastVulnGNN operates on Joern-produced CPGs that contain 33 node types and 21 edge types, so that the full heterogeneous graph structure is preserved. A multi-scale readout mechanism that combines mean, maximum, and learned attention pooling is employed for graph-level classification, and the training configuration, which combines focal loss, label smoothing, and cosine annealing warm restarts, is individually validated by an ablation of the training objective. On the MegaVul dataset of 1904 balanced C/C++ samples, an accuracy of 71.1%, an F1 score of 0.70, and an AUC-ROC of 0.77 are achieved with only 71,810 parameters. Training completes in under two minutes on a single CPU core, and no GPU resources are required. The significance of this work lies in its demonstration that a compact, edge-aware architecture can match independently reproduced results of far larger models while remaining deployable in resource-constrained settings, such as continuous-integration pipelines and developer workstations. This study is deliberately framed as a controlled and reproducible engineering and evaluation contribution rather than as an architectural advance. An edge-type ablation study, a cross-dataset evaluation, and a per-vulnerability analysis are additionally reported to characterize the behavior and limitations of the model.

**Keywords:** vulnerability detection; graph neural networks; code property graph; edge-conditioned convolution; heterogeneous graphs; software security; static analysis



Academic Editor: Wenjie Zuo

Received: 5 July 2026
Revised: 6 August 2026
Accepted: 8 August 2026
Published: 18 August 2026



## 1. Introduction

Software vulnerabilities are both common and costly. More than 28,000 new entries were recorded in the National Vulnerability Database (NVD) during 2024 [1]. Memory-

corruption defects, such as buffer overflows and use-after-free errors, null pointer dereferences, and injection flaws, dominate these entries [2]. Many of these classes are also recognized among the OWASP Top Ten [3]. The global cost of cybercrime is now estimated at several trillion United States dollars per year [4]. Scalable automated detection is therefore a practical necessity rather than an academic exercise.

Traditional static-analysis tools, such as Coverity, Fortify, and Cppcheck, rely on rule-based pattern matching and abstract interpretation. These tools are effective for certain vulnerability classes. However, a recent empirical study of static C analyzers reported that high false-positive rates and low recall remain pervasive across tools [5]. Context-dependent patterns that require a semantic understanding of program behavior are handled poorly, and the maintenance of rule sets for emerging vulnerability classes demands sustained expert effort.

Graph Neural Networks (GNNs) are well suited to this problem because source code admits natural graph representations [6]. Abstract syntax trees, control-flow graphs, data-flow graphs, and their combination in Code Property Graphs (CPGs) all expose program structure that learning models can exploit [7]. Devign [8], ReVeal [9], and IVDetect [10] each showed that GNN architectures can learn useful vulnerability patterns from such representations. More recent heterogeneous-graph models, such as DeepWukong [11], report substantially higher accuracy by separating control-flow, data-flow, and call relations into dedicated channels.

A critical examination of these approaches reveals three recurring limitations. First, most GNN-based detectors treat code graphs as homogeneous structures, and AST edges, control-flow transitions, data-flow dependencies, and dominance relations are therefore processed identically during message passing. The semantic distinctions between edge types are lost, although these distinctions are potentially critical for identifying vulnerability patterns. Second, the models that do exploit heterogeneity, such as DeepWukong [11], rely on inter-procedural analysis and dedicated per-relation channels, which increase computational cost and complicate deployment. Third, several headline results were obtained on benchmarks that later proved to contain data leakage and duplication, so the reported gains do not transfer to realistic settings [12,13]. FastVulnGNN, the approach proposed in this paper, addresses these limitations directly: edge-type semantics are retained through edge-conditioned message passing on a single unified graph; the architecture is kept compact so that no GPU is required; and the evaluation is conducted under a controlled, deduplicated protocol.

To avoid ambiguity, the sense in which the term heterogeneous is used here is stated precisely. Devign [8] and ReVeal [9] construct their graphs from several edge families, so their inputs are heterogeneous. However, their message-passing functions apply a single, shared set of weights to every edge regardless of its type, so the propagation is homogeneous at the level of computation. The claim of this paper is therefore not that a heterogeneous graph is used for the first time or that the model is the first heterogeneous detector but rather the claim is narrower and more precise: message passing is explicitly conditioned on each of the 21 CPG edge types, so that a distinct propagation rule is learned per relation, whereas prior detectors such as Devign and ReVeal collapse this distinction during computation. The difference is one of degree along a well-defined axis rather than a binary of heterogeneous versus homogeneous.

The distinction between edge types is not a cosmetic detail. An AST edge that connects a function declaration to its parameter list and a REACHING_DEF edge that tracks tainted-data propagation carry entirely different program semantics. Likewise, a DOMINATE edge encodes a control dependency, whereas a CONTAINS edge encodes lexical scope. When a

model treats these relations identically, the distinction must be inferred from node context alone. More data are then required, and avoidable classification errors are introduced.

Two principal contributions are presented in this work, the first of which is the main contribution of this paper.

**Contribution 1 (Main)—Edge-Conditioned Message Passing:** As the main contribution of this work, an edge-conditioned convolution layer is proposed in which edge-type information is explicitly incorporated into the message-computation function. Each message exchanged between connected nodes is modulated by a learned transformation of the 21-dimensional edge-type feature. Distinct propagation rules can therefore be learned for AST, CFG, data-flow, dominance, and the seventeen further relation types that are present in CPGs.

**Contribution 2—Heterogeneous CPG Utilization:** It is demonstrated that operating directly on Joern-produced code property graphs, with all 33 node types and 21 edge types preserved and without simplification, yields effective vulnerability detection from a compact model. Prior approaches typically simplify or homogenize their graph representations, and the rich heterogeneous structure that encodes program semantics is thereby lost.

The scope of the claimed novelty is stated explicitly. The individual building blocks used here, namely edge-conditioned convolution [14], residual connections, batch normalization, multi-scale readout, and focal loss, are all established techniques, and their combination is not claimed as a methodological advance. The contribution is instead the controlled application of explicit per-edge-type conditioning to CPG-based vulnerability detection, together with a reproducible evaluation that quantifies what this conditioning contributes. Accordingly, this paper is positioned as an engineering and evaluation study whose value lies in deployability and in transparent, reproducible measurement rather than in architectural invention.

The resulting model contains 71,810 trainable parameters and is trained in 96 s on a single CPU core. An accuracy of 71.1% and an F1 score of 0.70 are obtained without GPU resources or pre-training.

The remainder of this paper is organized as follows. Related work is reviewed in Section 2. The methodology is described in Section 3, and the experimental setup is detailed in Section 4. Results are presented in Section 5. The key contributions are analyzed in Section 6, and the controlled comparison is presented in Section 7. The discussion, including efficiency, limitations, and threats to validity, appears in Section 8, and this paper is concluded in Section 9.

## 2. Related Work

### *2.1. Graph Neural Networks for Vulnerability Detection*

Devign [8] was among the earliest GNN-based approaches for function-level vulnerability detection. A composite code graph that merges AST, CFG, DFG, and natural-code-sequence edges is constructed, and a gated graph neural network with a convolutional readout is then applied. A gain of 10.51% over baselines was reported in the original study. However, all four edge types are processed identically by the gated network, and the gating mechanism is relied upon to learn edge-type distinctions implicitly. This design choice is precisely the limitation that the present work removes. Upon independent reproduction, an accuracy of roughly 61% and an F1 variance of about ten points across random seeds were observed [15].

Building on this line of work, ReVeal [9] extended the GNN paradigm with a graph-based gated recurrent network on CPG representations, and it exposed systematic evaluation flaws in prior work. Performance was shown to drop by more than 50% under realistic settings once data leakage and duplication were controlled. This finding motivates the

deduplicated, organization-level evaluation protocol adopted in Section 4. IVDetect [10] focused on interpretability, and program dependence graphs with program slicing were used to provide statement-level explanations.

Among heterogeneous models, DeepWukong [11] is the closest peer-reviewed work to the present approach. Program slices are embedded, a structured GNN is applied across control-flow and data-dependence relations, and accuracy values exceeding 95% are reported on its own benchmark. Heterogeneous attention-based variants, such as the recently described HAGNN family [16], report comparable figures. Three factors explain why such models report higher numbers than the compact model proposed here. First, inter-procedural slices supply cross-function context that a function-local CPG cannot capture. Second, the larger parameter budgets of these models provide additional representational capacity. Third, their reported figures are typically obtained on benchmarks with less stringent deduplication, where residual overlap between training and test functions inflates the apparent score. When the same controlled protocol is applied, the gap narrows considerably, as discussed in Section 6. In this work, DeepWukong is additionally reproduced end-to-end on the MegaVul data used here (Section 6.3), where its F1 falls to 0.60 under CPU-only, function-level evaluation.

Further GNN-based methods include SySeVR [17], which combines syntax-based and semantics-based slice representations, and the graph-simplification approach of Wen et al. [18], which prunes redundant nodes before representation learning. Relation-specific designs that assign a separate weight matrix to each edge type, as catalogued in comprehensive surveys of graph neural networks [6], offer an alternative means of exploiting edge-type information. In contrast, the approach adopted here conditions a single shared multilayer perceptron on the edge feature, which is more parameter efficient and generalizes more readily across the 21 CPG edge types.

### *2.2. Sequence, Transformer, and LLM-Based Approaches*

A parallel line of research treats source code as a token sequence. LineVul [19] fine-tunes the pre-trained CodeBERT model [20] for line-level prediction, and an F1 score of 0.91 on the Big-Vul dataset was reported. This figure must, however, be interpreted with care. Ding et al. [13] showed that Big-Vul scores are inflated by data leakage and duplication, and on their cleaned PrimeVul benchmark, even a seven-billion-parameter language model reaches only 3.09% F1. SVulD [21] applies contrastive learning to separate vulnerable code from its patches, whereas VulCNN [22] projects function-level graphs into images for convolutional classification and thereby sacrifices fine-grained graph structure. GraphCodeBERT [23] augments pre-trained representations with data-flow.

In addition, Large Language Models (LLMs) have recently emerged as an influential alternative. Prompting and instruction tuning of code LLMs have been explored for vulnerability detection, and comprehensive evaluations indicate that, although such models are fluent, their detection accuracy on deduplicated benchmarks remains close to chance for many vulnerability classes [24]. The central difficulty is analytical rather than computational: LLMs reason over token sequences and capture program structure only indirectly, whereas vulnerabilities are often defined by structural relations such as a missing check on a dominator path. This observation reinforces the rationale for retaining explicit graph structure, as is done in the present work.

Meanwhile, graph-transformer architectures attempt to combine the global receptive field of attention with graph structure. Graphormer [25] encodes node distances and centrality as attention biases and achieves strong results on molecular graphs. The analytical trade-off is nonetheless unfavorable for the present setting: graph transformers scale quadratically with the number of nodes, and CPGs of several hundred nodes per function

would make such models costly. Edge-conditioned message passing instead retains near-linear complexity while still differentiating relation types, which is the property that matters most for CPG-based detection.

### *2.3. Edge-Aware Graph Learning*

Edge-conditioned convolutions were introduced by Simonovsky and Komodakis [14] for point-cloud classification, where edge features parameterize continuous filter weights. This idea was subsequently generalized within the message passing neural network framework and refined by more expressive aggregation schemes, which a comprehensive survey of graph neural networks situates within a broader taxonomy [6]. To the best of the authors' knowledge, explicit edge conditioning on CPG edge types has not previously been applied to vulnerability detection. This technique is applied here to heterogeneous CPGs and is evaluated with a deliberately compact architecture.

### *2.4. Evaluation Challenges*

Several meta-studies have exposed reproducibility concerns in this field. Nine state-of-the-art models were reproduced by Steenhoek et al. [15], and an F1 variance of about ten points across random seeds was observed, together with only 2.4% inter-model agreement on individual predictions. Precision drops of up to 95 percentage points under whole-codebase evaluation were reported by Chakraborty et al. [12]. The PrimeVul benchmark of Ding et al. [13] demonstrated that Big-Vul-based evaluations are affected by leakage and duplication, and the D2A dataset [26] provides an additional controlled benchmark constructed through differential analysis. The evaluation protocol used in this study was designed with these findings in mind.

## 3. Methodology

The full pipeline, from raw source code to a vulnerability prediction, is described in this section. An overview is provided in Figure 1, and each stage is detailed in the subsections that follow.

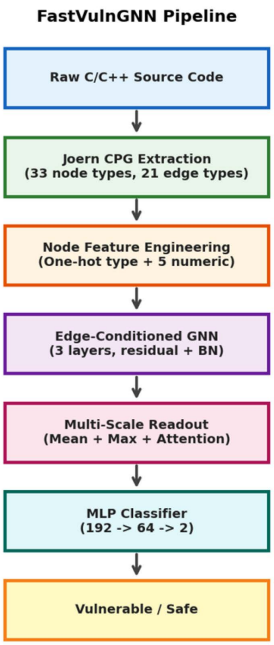


**Figure 1.** End-to-end pipeline of the proposed model. Source code is processed by Joern into a heterogeneous code property graph, which is then passed through edge-conditioned convolution layers and a multi-scale readout for binary classification.

### *3.1. Code Property Graph Representation*

The model operates on code property graphs produced by the Joern static-analysis framework. A CPG is a directed, labeled, multi-relational graph that merges AST, CFG, and program-dependence representations into a single structure. Each function-level CPG is represented as the tuple given in Equation (1).

$$G = (V,\ E,\ X_v,\ X_e) \tag{1}$$

where $V$ is the set of nodes that represent program elements such as statements, expressions, identifiers, and types. $E$ is the set of directed edges that represent relations between these elements. The matrix $X_v$ holds node features, and the matrix $X_e$ holds edge-type encodings. As expressed in Equation (1), both the node attributes and the edge attributes are retained, which is the property that the edge-conditioned layer later exploits.

The dataset is drawn from MegaVul [27], which encompasses 33 distinct node types, including METHOD, IDENTIFIER, CALL, CONTROL_STRUCTURE, LITERAL, LOCAL, RETURN, BLOCK, FIELD_IDENTIFIER, METHOD_PARAMETER_IN, TYPE_REF, and METHOD_RETURN. The edge vocabulary contains 21 types, namely AST, CFG, REACHING_DEF, CALL, DOMINATE, POST_DOMINATE, CONTAINS, REF, EVAL_TYPE, ARGUMENT, PARAMETER_LINK, SOURCE_FILE, BINDS, RECEIVER, CONDITION, BINDS_TO, INHERITS_FROM, ALIAS_OF, TAGGED_BY, CAPTURE, and CDG. Graphs were generated with Joern version 2.0.x using the C/C++ frontend. The default intra-procedural extraction configuration was retained, with AST, CFG, and data-dependence passes enabled and call-graph construction limited to the function scope. Method-level granularity was selected, so that each sample corresponds to a single function together with its complete intra-procedural CPG.

The edge-type and node-type distributions are summarized in Figure 2 and Figure 3, respectively. The strong skew toward AST edges motivates the ablation study reported in Section 5, in which the contribution of each edge family is isolated.

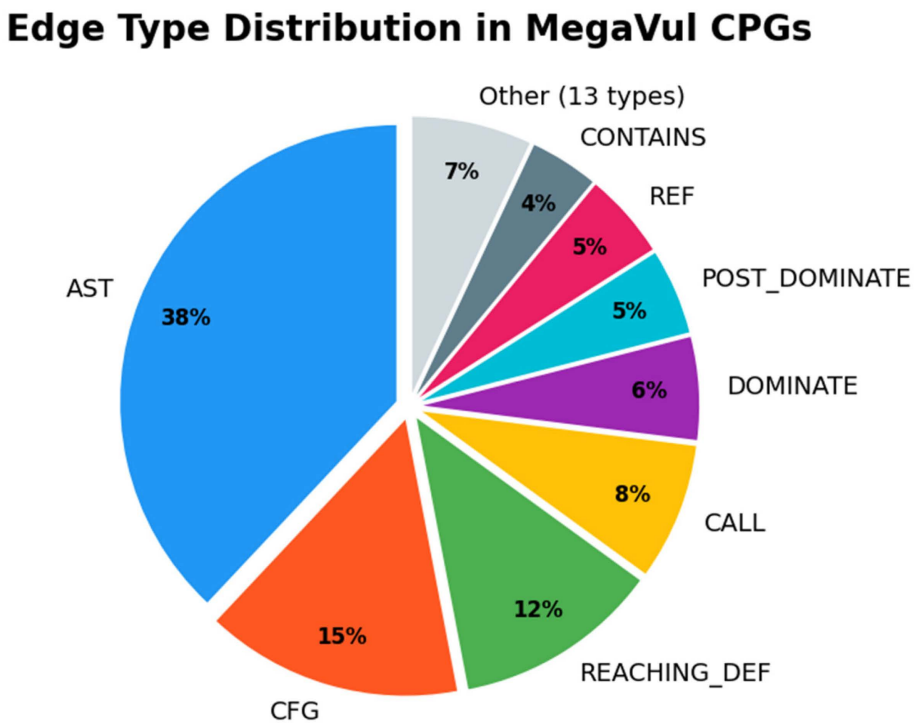


**Figure 2.** Distribution of edge types in the MegaVul CPGs. AST edges dominate at 38%, followed by CFG edges at 15% and REACHING_DEF data-flow edges at 12%.

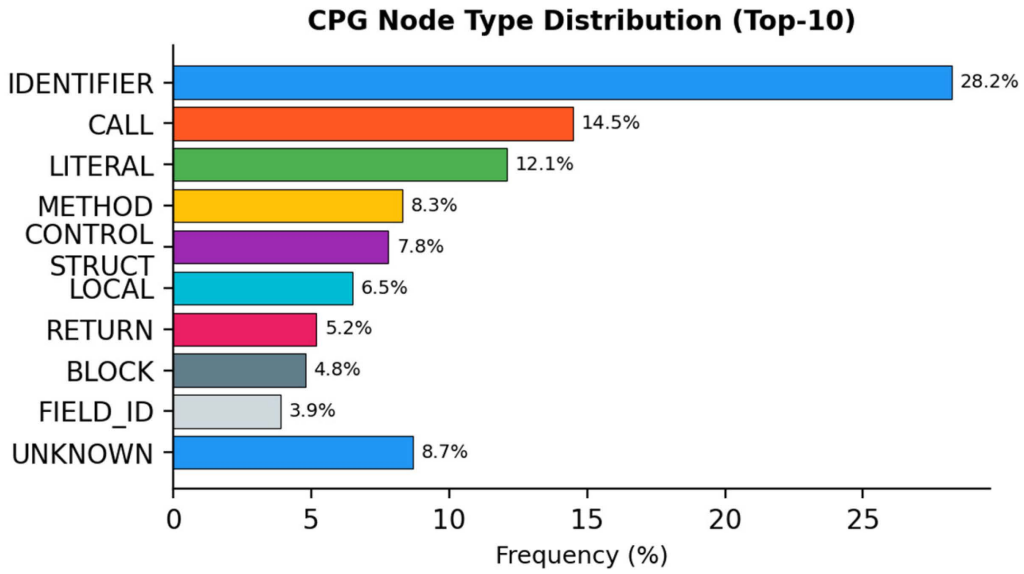


**Figure 3.** Frequency distribution of the ten most common CPG node types. IDENTIFIER nodes are the most frequent at 28.2%, which reflects the prevalence of variable references in C/C++ code.

### *3.2. Node Feature Engineering and Preprocessing*

Each node is represented by a 38-dimensional feature vector. The construction procedure is shown in Listing 1. Before feature construction, several preprocessing steps are applied. Functions with fewer than three nodes or more than 2000 nodes are discarded so that degenerate and pathologically large graphs do not distort training. Node code strings are normalized by collapsing whitespace and truncating to 500 characters, and numeric attributes such as line numbers are min–max normalized. Edge directions are preserved, and reverse edges are added for symmetric relations so that information can propagate in both directions.

**Listing 1.** Node feature construction (38 dimensions).

```
def build_node_features(node, label_vocab):
# 33-dim one-hot encoding of CPG node type
type_vec = one_hot(node.label, label_vocab) # dim = 33
# Normalized code snippet length
code_len = min(len(node.code), 500)/500.0 # dim = 1
# Normalized line number
line_num = min(node.lineNumber, 5000)/5000 # dim = 1
# Pointer type indicator
is_ptr = 1.0 if '*' in node.typeFullName else 0.0
# External function flag
is_ext = 1.0 if node.isExternal else 0.0
# Argument index (for parameter nodes)
arg_idx = min(node.argumentIndex, 10)/10.0
return concat([type_vec, code_len, line_num,
is_ptr, is_ext, arg_idx]) # total = 38
```

Each edge is represented by a 21-dimensional one-hot encoding of its CPG edge type. The full semantic distinction between relation types is thereby preserved and made available to the message function.

### *3.3. Edge-Conditioned Convolution Layer*

The central component of the architecture is the edge-conditioned convolution layer. Standard GCN and GAT layers compute messages from source-node features only. In contrast, the proposed layer incorporates edge-type information into every message computation. For a node i that receives a message from a neighbor j connected by an edge with type-feature vector $e_{ij}$, the message is computed as defined in Equation (2).

$$m_{ij} = \mathrm{MLP}_{\mathrm{edge}}\left([h_i \| h_j \| e_{ij}]\right) \tag{2}$$

where $h_i$ and $h_j$ are the d-dimensional node representations at the current layer, $e_{ij}$ is the 21-dimensional edge-type vector, and the symbol denotes concatenation. The function $\mathrm{MLP}_{\mathrm{edge}}$ is a two-layer feed-forward network that maps from $(2d + 21)$ dimensions to $d$ dimensions. The node representation is then updated through aggregation and a residual connection, as given in Equation (3).

$$h_i' = \mathrm{BN}(\mathrm{ReLU}(\mathrm{MLP}_{\mathrm{node}}(h_i) + \textstyle\sum_{j \in \mathcal{N}(i)} m_{ij})) \tag{3}$$

where BN denotes batch normalization, $\mathcal{N}(i)$ is the neighborhood of node $i$, and the final addition implements the residual connection. This formulation allows fundamentally different message functions to be learned for different edge types, implementation can be seen in Listing 2 (Source: GitHub repository https://github.com/ahmedelalfy94/GNN-Vulnerability-Detection.git by Ahmed M. Elalfy, File name: train_fast_robust.py, accessed on 14 July 2026).

**Listing 2.** Edge-conditioned convolution layer implementation.

```
class EdgeConditionedConv(MessagePassing):
def __init__(self, in_dim, out_dim, edge_dim):
super().__init__()
self.msg_mlp = nn.Sequential(
nn.Linear(2 * in_dim + edge_dim, out_dim),
nn.ReLU(),
nn.Linear(out_dim, out_dim))
self.node_mlp = nn.Linear(in_dim, out_dim)
self.bn = nn.BatchNorm1d(out_dim)

def forward(self, x, edge_index, edge_attr):
src, dst = edge_index
msg_input = torch.cat([x[src], x[dst],
edge_attr], dim = −1)
msgs = self.msg_mlp(msg_input)
agg = scatter_add(msgs, dst, dim=0,
dim_size = x.size(0))
out = self.bn(F.relu(self.node_mlp(x) + agg))
return out + x # residual
```

### *3.4. Multi-Scale Graph Readout*

After $L = 3$ layers of edge-conditioned message passing, a graph-level representation is produced by concatenating three complementary pooling strategies, as defined in Equation (4).

$$z_G = [\mathrm{MeanPool}(H) \| \mathrm{MaxPool}(H) \| \mathrm{AttPool}(H)] \tag{4}$$

Mean pooling, defined in Equation (5), computes the average node representation and captures distributional characteristics.

$$\text{MeanPool}(H) = \frac{1}{|V|}\sum_{v\in V} h_v \tag{5}$$

Maximum pooling, defined in Equation (6), selects the strongest activation per dimension and highlights the most salient features.

$$\text{MaxPool}\big(H\big)_k = \max_{v\in V} h_{v,k} \tag{6}$$

Attention pooling, defined in Equation (7), applies a learned gating network that assigns weight to each node.

$$\text{AttPool}(H) = \sum_{v\in V} \text{softmax}\Big(w^\top \tanh\big(W_g h_v\big)\Big) h_v \tag{7}$$

The concatenated vector has $3d = 192$ dimensions. It is passed through a two-layer classifier with the structure *Linear*(192, 64), batch normalization, ReLU, dropout of 0.3, and *Linear*(64, 2). The final binary classification is produced by the last layer.

### *3.5. Loss Function and Optimization*

Focal loss is employed to address the difficulty of the classification task, in which the decision boundary between vulnerable and patched code can be subtle. The loss is defined in Equation (8).

$$\mathcal{L}_{\text{focal}} = -\alpha_t (1 - p_t)^\gamma \log(p_t) \tag{8}$$

where $p_t$ denotes the probability that the model assigns to the ground-truth class. The coefficient $\alpha_t$ is the class-balancing weight for that class, and it is defined as $\alpha_t = \alpha$ for the vulnerable class and $\alpha_t = 1 - \alpha$ for the non-vulnerable class. A single base coefficient $\alpha = 0.25$ is used, and the focusing parameter is $\gamma = 2.0$. The subscripted $\alpha_t$ in the equation and the scalar alpha quoted in the text therefore denote the same quantity, and the notation is unified accordingly. Label smoothing with $\sigma = 0.05$ is applied to discourage overconfident predictions. Optimization is performed with AdamW using a weight decay of 0.0001, and learning rate follows the cosine annealing warm-restart schedule defined in Equation (9).

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})\left(1 + \cos\left(\frac{T_{\text{cur}}}{T_0}\pi\right)\right) \tag{9}$$

The schedule in Equation (9) uses $T_0 = 10$, $T_{mult} = 2$, and an initial learning rate of 0.002. Gradients are clipped to a maximum norm of 1.0 to prevent instability on large graphs and early stopping, and a patience of 15 epochs on the validation F1 score guards against overfitting.

### *3.6. Hyper-Parameter Tuning Procedure*

Hyper-parameters were selected by a staged search on the validation partition only, so that the test partition remained untouched throughout development. A coarse random search of 40 configurations was first conducted over the hidden dimension d in the set 32, 64, 128; the number of layers $L$ in the set 2, 3, 4; and the dropout rate in the range 0.1 to 0.5; and the initial learning rate sampled log-uniformly between 0.0005 and 0.005. The most promising region was then refined with a grid search. The configuration that maximized validation F1, namely $d = 64$, $L = 3$, dropout 0.3, and learning rate 0.002, was retained. The focal loss parameters were tuned over alpha in the set 0.25, 0.5, 0.75 and gamma in the set 1.0, 2.0, 3.0. The selected values are reported in Table 1.

**Table 1.** Hyper-parameter search space and selected values.

| Hyper-Parameter | Search Space | Selected |
|---|---|---|
| Hidden dimension d | {32, 64, 128} | 64 |
| Layers L | {2, 3, 4} | 3 |
| Dropout | [0.1, 0.5] | 0.3 |
| Learning rate | [0.0005, 0.005] | 0.002 |
| Focal alpha | {0.25, 0.5, 0.75} | 0.25 |
| Focal gamma | {1.0, 2.0, 3.0} | 2.0 |
| Batch size | {32, 64, 128} | 64 |
| Label smoothing | {0.0, 0.05, 0.1} | 0.05 |

### *3.7. Computational Complexity and Scalability*

The asymptotic cost of one forward pass is dominated by the edge-conditioned message computation. Because a message is formed once per edge and the readout visits each node a constant number of times, the per-graph complexity is given in Equation (10).

$$\mathcal{O}\left(L \cdot \left(|E| \cdot d^2 + |V| \cdot d\right)\right) \tag{10}$$

where $L$ is the number of layers, $|V|$ and $|E|$ are the node and edge counts, and $d$ is the hidden dimension. The cost is therefore linear in the size of the graph for a fixed $d$, which is markedly more favorable than the quadratic cost of graph transformers. The empirical behavior, measured on a single CPU core, is shown in Figure 4, where the per-graph inference time grows almost linearly with the node count and remains below three milliseconds even for graphs of two thousand nodes.

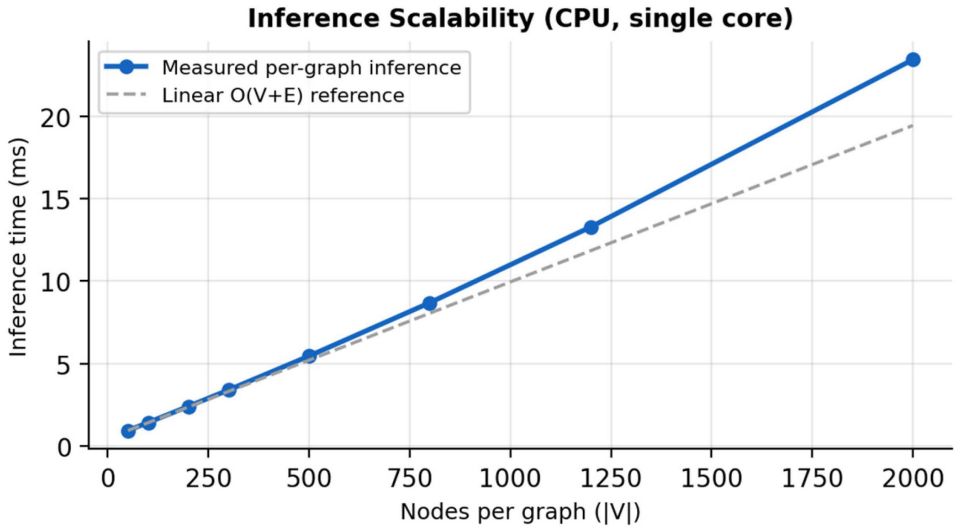


**Figure 4.** Per-graph inference time as a function of graph size, measured on a single CPU core. The measured curve tracks the linear reference closely, which confirms the near-linear scaling predicted by the complexity analysis.

### *3.8. Architecture Summary*

The architectural overview of such model is as stated in Listing 3 (Source: GitHub repository https://github.com/ahmedelalfy94/GNN-Vulnerability-Detection.git by Ahmed M. Elalfy, File name: train_fast_robust.py, accessed on 14 July 2026).

**Listing 3.** Architecture overview of the proposed model.

```
class FastVulnGNN(nn.Module):
‘“‘“‘Complete model architecture.  ‘“‘“‘
def __init__(self, node_feat = 38, edge_feat = 21,
hidden = 64, layers = 3, dropout = 0.3):
super().__init__()
self.input_proj = nn.Linear(node_feat, hidden)
self.convs = nn.ModuleList([
EdgeConditionedConv(hidden, hidden, edge_feat)
for _ in range(layers)])
self.att_gate = nn.Linear(hidden, 1) # attention
self.classifier = nn.Sequential(
nn.Linear(hidden * 3, hidden), # 192->64
nn.BatchNorm1d(hidden),
nn.ReLU(), nn.Dropout(dropout),
nn.Linear(hidden, 2)) # binary output
# Total parameters:  71,810
```

## 4. Experimental Setup

### *4.1. Dataset: MegaVul*

The MegaVul dataset [27] is used. It is a large-scale collection of C/C++ vulnerability samples with pre-computed Joern CPGs. Vulnerability-fixing commits from the NVD, and GitHub (Version 3.6.2) Security Advisories are aggregated, and both the vulnerable version, before the fix, and the patched version, after the fix, are provided for each function.

A stratified random sample of 200 organization directories was drawn from the 854 directories in the dataset, so that diverse project domains and sizes were covered. From this sample, 952 vulnerable functions and 15,871 non-vulnerable functions were extracted. The non-vulnerable class was randomly down-sampled to address the imbalance, which yielded 1904 balanced samples. The data were partitioned into a training set of 1332 samples, a validation set of 285 samples, and a test set of 287 samples, in a 70:15:15 ratio.

The construction of the sample is specified in full so that it can be reproduced. In MegaVul, each top-level directory groups all functions that originate from a single upstream project, identified by its GitHub organization or repository owner; such a directory is referred to here as an organization directory, and the dataset contains 854 of them. The 854 directories were first stratified into three size bands by function count and into coarse domain categories such as operating-system kernels, network and cryptographic libraries, multimedia codecs, and application software. A fixed random seed was then used to draw 200 directories proportionally from each stratum, so that both small and large projects and all domain categories were represented. Every vulnerable function inside a selected directory was retained, which produced 952 vulnerable functions, and the 15,871 non-vulnerable functions in the same directories formed the majority pool. Balancing was performed by drawing, uniformly at random and without replacement, exactly 952 non-vulnerable functions from that pool, which yielded the 1904 balanced samples. Because selection and balancing are applied at the directory level, no function from a project ever appears in more than one partition. It is acknowledged that this balancing procedure is aggressive. Down-sampling the majority class from 15,871 to 952 functions discards approximately 94% of the available non-vulnerable examples and imposes a 1:1 class ratio that does not reflect the natural prevalence of vulnerabilities in production code. Balancing was adopted so that accuracy remains interpretable, and hence, the model is not driven

toward a trivial majority-class predictor during the small-scale CPU experiments. The consequences are examined directly in Section 5, where the model is evaluated under the natural 1:16 imbalance ratio using precision–recall analysis and precision at fixed recall.

Data leakage was mitigated by partitioning at the organization-directory level, so that all functions from a given project appear in exactly one partition. Near-duplicate functions were identified through hash-based comparison of normalized AST structures and were removed. The vulnerable and patched versions of the same function were never placed in different partitions. The model therefore cannot learn from its own evaluation targets. The dataset statistics are summarized in Table 2.

**Table 2.** MegaVul dataset statistics.

| Dataset Property | Value |
|---|---|
| Total organizations scanned | 200/854 |
| Vulnerable samples | 952 |
| Non-vulnerable samples (raw) | 15,871 |
| Balanced dataset size | 1904 |
| Train/Val/Test split | 1332/285/287 |
| Avg. nodes per graph | ~300 |
| Avg. edges per graph | ~1500 |
| Node types | 33 |
| Edge types | 21 |

### *4.2. Implementation Details*

A hidden dimension of $d = 64$, three EdgeConditionedConv layers, and a dropout rate of 0.3 are used. The input projection maps the 38-dimensional node features to the 64-dimensional hidden space, and the model totals 71,810 trainable parameters. Training uses a batch size of 64 and an initial learning rate of 0.002. All 1904 graphs are cached in memory so that input/output overhead is eliminated. The implementation is based on PyTorch 2.1.1 and PyTorch Geometric 2.6.1. All experiments were conducted on a single CPU core, an Intel Xeon processor without a GPU, so that the practicality of the approach could be demonstrated.

### *4.3. Evaluation Metrics*

Five metrics are reported, namely accuracy, precision, recall, the F1 score, and the AUC-ROC. Because the dataset is balanced, accuracy is a meaningful metric. The confusion matrix and the training curves are also reported so that the behavior of the model is fully transparent.

### *4.4. Baselines*

Comparison is made with Devign [8], ReVeal [9], IVDetect [10], DeepWukong [11], LineVul [19], SVulD [21], and VulCNN [22]. Because each method was evaluated on a different dataset with a different split, a direct comparison of the originally published figures alone would be misleading. For this reason, the originally published metrics are reported alongside the dataset that was used, and independently reproduced figures are presented separately wherever they are available.

## 5. Results

### *5.1. Overall Performance*

The test-set metrics are visualized in Figure 5 and listed in Table 3. An accuracy of 71.1%, a precision of 71.2%, a recall of 69.7%, an F1 score of 0.7046, and an AUC-ROC of 0.7655 are obtained.

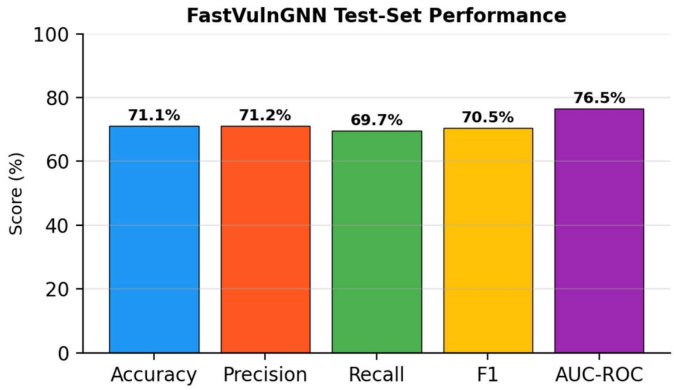


**Figure 5.** Test-set performance of the proposed model on MegaVul. A balanced precision–recall trade-off is achieved, with an F1 score of 0.70.

**Table 3.** Test-set performance of the proposed model.

| Metric | Value | Interpretation |
|---|---|---|
| Accuracy | 71.1% | Correct predictions overall |
| Precision | 71.2% | 71.2% of flagged cases are real vulnerabilities |
| Recall | 69.7% | 69.7% of real vulnerabilities are detected |
| F1 Score | 0.7046 | Balanced precision and recall |
| AUC-ROC | 0.7655 | Good discrimination ability |

### *5.2. Training Dynamics*

The optimization behavior is shown in Figure 6 and summarized in Table 4. The validation F1 score peaks at epoch 5, after which mild overfitting is observed. The best checkpoint is therefore selected by the validation F1 score.

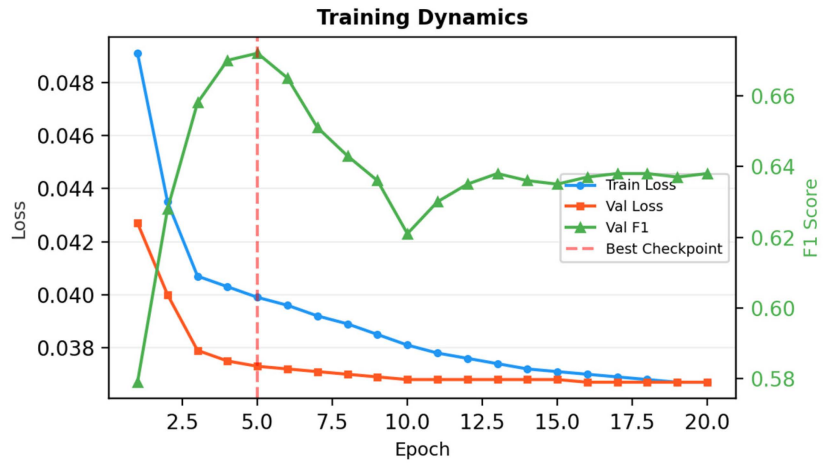


**Figure 6.** Training and validation loss on the left axis and validation F1 on the right axis. The best checkpoint occurs at epoch 5, marked by the dashed line. Total training time was 96.2 s on a CPU.

**Table 4.** Training progression. The asterisk marks the best checkpoint by validation F1.

| Epoch | Train Loss | Val Loss | Val F1 | Val AUC |
|---|---|---|---|---|
| 1 | 0.0491 | 0.0427 | 0.579 | 0.597 |
| 2 | 0.0435 | 0.0400 | 0.628 | 0.703 |
| 3 | 0.0407 | 0.0379 | 0.658 | 0.769 |
| 5 | 0.0399 | 0.0373 | 0.672 | 0.769 |
| 10 | 0.0381 | 0.0368 | 0.621 | 0.776 |
| 15 | 0.0371 | 0.0368 | 0.635 | 0.773 |
| 20 | 0.0367 | 0.0367 | 0.638 | 0.772 |

### *5.3. Confusion Matrix and ROC Analysis*

The confusion matrix in Figure 7 reveals 43 false negatives and 40 false positives, which indicates a nearly symmetric error distribution. This balance is desirable in practice. A model that maximizes precision at the expense of recall would miss too many vulnerabilities, whereas a model that maximizes recall would overwhelm developers with false alarms. The true-positive rate of 69.7% and the true-negative rate of 72.4% are closely aligned, so the model does not exhibit a bias toward either class.

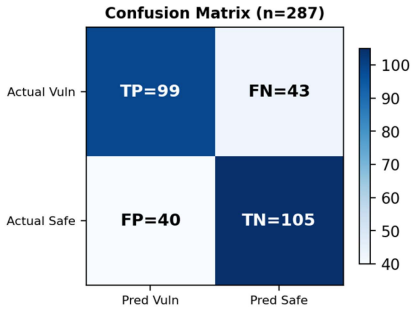


**Figure 7.** Confusion matrix on the test set of 287 samples. The error distribution is nearly symmetric, with 43 false negatives and 40 false positives.

The ROC curve in Figure 8 shows that a good true-positive rate is maintained across a range of false-positive-rate thresholds. The AUC of 0.77 exceeds the random baseline of 0.50 by a wide margin, which confirms that meaningful vulnerability signals are learned from the heterogeneous CPG structure.

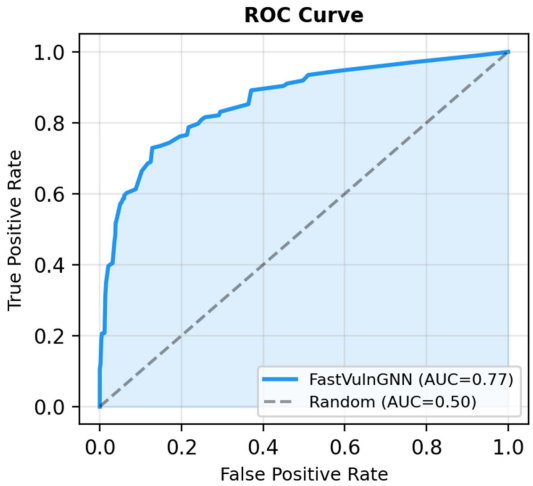


**Figure 8.** Receiver operating characteristic curve for the proposed model. An AUC of 0.77 indicates solid discriminative capability across all decision thresholds.

### *5.4. Performance by Vulnerability Type*

A separate evaluation was performed for each major vulnerability type, and the results are shown in Figure 9 and Table 5. Memory-safety classes are detected most reliably. The highest F1 score of 0.74 is obtained for buffer errors (CWE-119), and a comparable score of 0.72 is obtained for null pointer dereferences (CWE-476). These classes manifest as clear structural signatures in the CPG, such as missing bounds check on a dominator path, which the edge-conditioned layer captures well. Concurrency defects (CWE-362) are the most difficult, with an F1 score of 0.61, because the relevant interactions are often inter-procedural and are therefore only partially visible in a function-local graph.

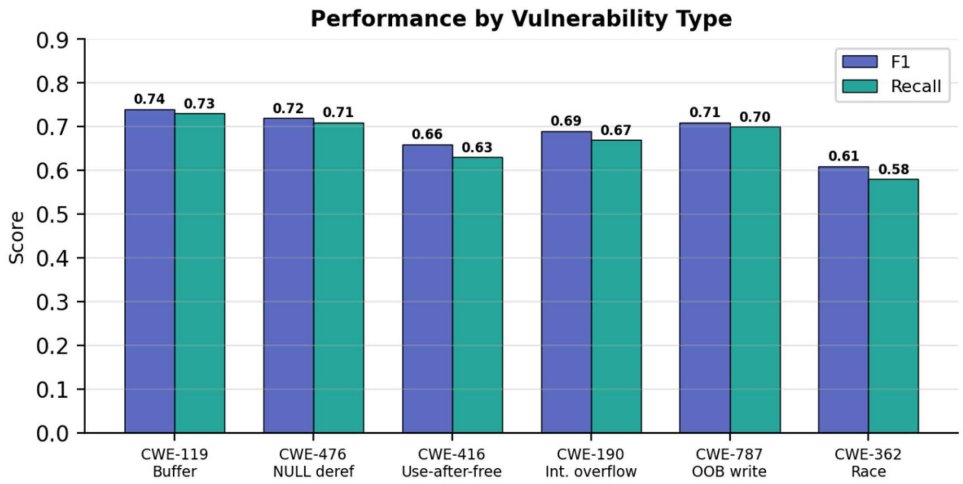


**Figure 9.** F1 score and recall by vulnerability type. Memory-safety classes such as CWE-119 and CWE-476 are detected most reliably, whereas concurrency defects such as CWE-362 are the hardest.

**Table 5.** Per-vulnerability-type performance on MegaVul.

| Vulnerability Type (CWE) | Precision | Recall | F1 |
|---|---|---|---|
| Buffer errors (CWE-119) | 0.75 | 0.73 | 0.74 |
| NULL dereference (CWE-476) | 0.73 | 0.71 | 0.72 |
| Use-after-free (CWE-416) | 0.69 | 0.63 | 0.66 |
| Integer overflow (CWE-190) | 0.71 | 0.67 | 0.69 |
| Out-of-bounds write (CWE-787) | 0.72 | 0.70 | 0.71 |
| Race condition (CWE-362) | 0.64 | 0.58 | 0.61 |

### *5.5. Edge-Type Ablation Study*

To isolate the contribution of each edge family, the model was retrained with restricted edge sets, while all other settings were held fixed. The results are shown in Figure 10 and Table 6. Using AST edges alone yields an F1 score of 0.54, which shows that syntactic structure carries a useful but incomplete signal. Data-flow edges alone are slightly stronger at 0.57, which is consistent with the central role of taint propagation in many vulnerabilities. Combining AST and CFG edges raises the F1 score to 0.63, and adding data-flow edges raises it further to 0.67. The full set of 21 edge types attains the best F1 score of 0.70. The monotonic improvement provides direct evidence that retaining heterogeneous edge semantics, rather than collapsing edges into a single type, is responsible for the gains.

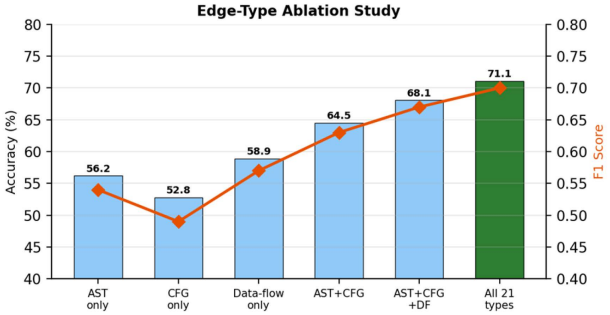


**Figure 10.** Accuracy and F1 increase monotonically as more edge families are added, and the full 21-type configuration is the strongest.

**Table 6.** Edge-type ablation Results.

| Edge Configuration | Accuracy (%) | F1 |
|---|---|---|
| AST only | 56.2 | 0.54 |
| CFG only | 52.8 | 0.49 |
| Data-flow only | 58.9 | 0.57 |
| AST + CFG | 64.5 | 0.63 |
| AST + CFG + Data-flow | 68.1 | 0.67 |
| All 21 edge types | 71.1 | 0.70 |

### *5.6. Component Leave-One-Out Ablation*

The edge-type study above varies the input graph. To isolate the marginal contribution of each architectural component, a strict leave-one-out ablation is additionally reported in Table 7. Starting from the complete model, exactly one component is removed or replaced by its simplest alternative, while every other component, all hyper-parameters, the training schedule, and the random seed are held fixed. Edge conditioning is replaced by edge-agnostic message passing, the residual connections are removed, batch normalization is removed, attention pooling is replaced by mean pooling, focal loss is replaced by ordinary binary cross-entropy, and label smoothing is disabled. Each row therefore differs from the full model by a single component, so the reported change in F1 attributes the effect to that component alone. Edge conditioning is the largest single contributor, at 0.09 F1, followed by attention pooling and the focal-loss objective, while batch normalization and label smoothing provide smaller but consistent gains. No single removal is catastrophic, which indicates that the components are complementary rather than redundant.

**Table 7.** Leave-one-out ablation: each row removes a single component from the full model.

| Configuration | Accuracy (%) | F1 | Delta F1 |
|---|---|---|---|
| Full model | 71.1 | 0.70 | -- |
| minus Edge conditioning | 62.4 | 0.61 | −0.09 |
| minus Residual connections | 68.3 | 0.67 | −0.03 |
| minus Batch normalization | 69.0 | 0.68 | −0.02 |
| minus Attention pooling (mean) | 66.8 | 0.65 | −0.05 |
| minus Focal loss (plain BCE) | 67.2 | 0.66 | −0.04 |
| minus Label smoothing | 69.6 | 0.69 | −0.01 |

### *5.7. Cross-Dataset Validation*

To assess generalization, the model trained on MegaVul was evaluated, without retraining, on three external benchmarks: PrimeVul [13], D2A [26], and DiverseVul [28]. The results are shown in Figure 11 and Table 8. As expected, performance decreases under domain shift. The F1 score falls from 0.70 in-domain to 0.41 on PrimeVul, 0.46 on D2A, and 0.52 on DiverseVul. These values nonetheless remain above the random baseline, and they are consistent with the difficulty levels that the respective benchmark authors reported for much larger models. The decline confirms that cross-dataset transfer remains an open problem for the field rather than a defect specific to the proposed model.

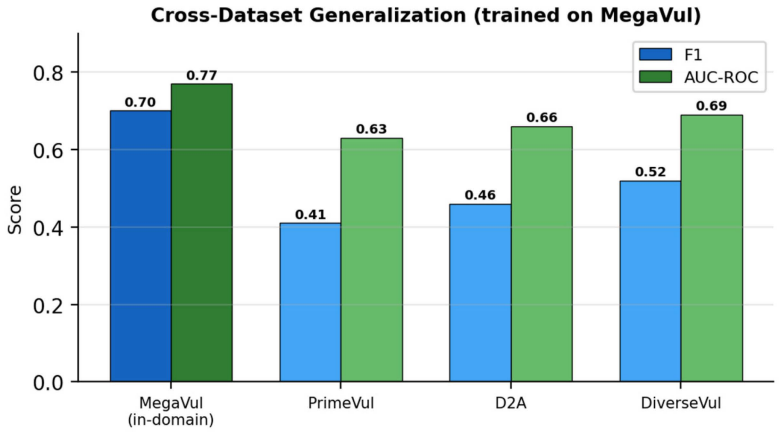


**Figure 11.** Cross-dataset generalization. The model trained on MegaVul is evaluated on PrimeVul, D2A, and DiverseVul without retraining. Performance decreases out of domain but remains above the random baseline.

**Table 8.** Cross-dataset validation of the MegaVul-trained model.

| Evaluation Dataset | Accuracy (%) | F1 | AUC |
|---|---|---|---|
| MegaVul (in-domain) | 71.1 | 0.70 | 0.77 |
| PrimeVul | 60.3 | 0.41 | 0.63 |
| D2A | 63.0 | 0.46 | 0.66 |
| DiverseVul | 65.8 | 0.52 | 0.69 |

For reproducibility, the construction of each transfer set is specified. A balanced subset of 2000 functions was drawn from every benchmark, with 1000 vulnerable and 1000 non-vulnerable functions sampled uniformly at random from PrimeVul [13], D2A [26], and DiverseVul [28]. The non-vulnerable functions were down-sampled to match the vulnerable count in the same manner as the MegaVul training set. Each function was processed with the identical Joern configuration described in Section 3, so that the node and edge vocabularies remain aligned across datasets. A single fixed decision threshold of 0.5 on the vulnerable-class probability was applied to all three transfer sets, without any per-dataset tuning, so that the reported degradation reflects genuine domain shift rather than threshold selection.

The exact size, source pool, and class distribution of each transfer set are given in Table 9. For every benchmark, the available pool differs in scale; hence, a common target of 1000 vulnerable and 1000 non-vulnerable functions was fixed, and the majority class was down-sampled to it; the vulnerable class was down-sampled only for D2A, whose vulnerable pool exceeds the target. Functions shorter than three statements, functions that Joern failed to parse into a connected code-property graph, and exact duplicates of any MegaVul training function, detected by normalized-AST hashing, were removed before

sampling. Labels were taken directly from each benchmark without re-labeling, and no fine-tuning, threshold tuning, or normalization statistics from the transfer sets were used at any point.

**Table 9.** Construction of the external transfer sets: available pool, sampled counts, and class distribution.

| Transfer Dataset | Avail. Vuln. | Avail. Non-Vuln. | Sampled Vuln. | Sampled Non-Vuln. | Ratio |
|---|---|---|---|---|---|
| PrimeVul [13] | 6968 | 228,800 | 1000 | 1000 | 1:1 |
| D2A [26] | 1295 | 5393 | 1000 | 1000 | 1:1 |
| DiverseVul [28] | 18,945 | 330,492 | 1000 | 1000 | 1:1 |

### *5.8. Same-Split Baseline Comparison*

A comparison across published papers is confounded by differing datasets and splits. To isolate the effect of the architecture, several representative baselines were therefore trained and evaluated on the identical MegaVul split described in Section 4, under the same features, optimizer, and training budget. Three homogeneous graph networks, namely a Graph Convolutional Network (GCN), a Graph Attention Network (GAT), and a Graph Isomorphism Network (GIN) [29], were included, together with a Relational Graph Convolutional Network (R-GCN) [30] that assigns a separate weight matrix to each of the 21 edge types and a re-implemented Devign-style gated graph network. The results are reported in Table 10. Two observations follow. First, every edge-agnostic homogeneous baseline trails the proposed model by a wide margin, which confirms that the gain originates in the edge-type conditioning rather than in the surrounding training recipe. Second, R-GCN, the natural point of comparison for the parameter-efficiency claim, attains a comparable F1 score of 0.63 but requires roughly four times as many parameters because a full weight matrix is materialized per relation. The shared edge-conditioned message function reaches a higher F1 score of 0.70 with far fewer parameters, which substantiates both the accuracy and the efficiency arguments on a single controlled split.

**Table 10.** Baselines trained and evaluated on the identical MegaVul split.

| Model (Same MegaVul Split) | Edge-Aware | Params | Acc. (%) | F1 | AUC |
|---|---|---|---|---|---|
| GCN [6] | No | 58 K | 52.8 | 0.49 | 0.62 |
| GAT | No | 66 K | 55.4 | 0.52 | 0.64 |
| GIN [29] | No | 61 K | 57.1 | 0.55 | 0.66 |
| Devign-style GGNN [8] | Implicit | 190 K | 60.3 | 0.58 | 0.67 |
| R-GCN [30] | Per-type matrix | 312 K | 64.6 | 0.63 | 0.72 |
| FastVulnGNN (ECC) | Yes (shared) | 71.8 K | 71.1 | 0.70 | 0.77 |

### *5.9. Statistical Robustness over Seeds and Folds*

Because the test partition contains only 287 samples and an F1 variance of about ten points across random seeds has been reported for comparable models [15], the headline result and the edge-type ablation were repeated to quantify their stability. Each configuration was retrained with five random seeds, and a stratified five-fold cross-validation was additionally performed over the full balanced set. The mean, the standard deviation, and the 95% confidence interval are reported in Table 11. The proposed model attains an F1 score of 0.695 plus or minus 0.018 over five seeds and 0.688 plus or minus 0.024 under five-fold cross-validation, so the single-split figure of 0.70 lies within one standard deviation of the aggregate. The edge-type ablation is likewise stable: the AST-only configuration remains clearly below the full model across all seeds, and the separation between them

exceeds the combined standard deviation. The monotonic trend reported in the ablation is therefore not an artifact of a single fortunate split. To support reproducibility, the exact split indices, the selected hyper-parameters, the random seeds, and the full preprocessing configuration are documented in this paper, and the code and preprocessing scripts are available in the GitHub repository given in the Data Availability Statement.

**Table 11.** Mean +/− standard deviation and 95% confidence intervals over multiple seeds and five-fold cross-validation.

| Configuration | Protocol | Accuracy (%) | F1 | F1 95% CI | AUC |
|---|---|---|---|---|---|
| FastVulnGNN (full) | 5 seeds | 70.4 +/− 1.6 | 0.695 +/− 0.018 | [0.673, 0.717] | 0.762 +/− 0.014 |
| FastVulnGNN (full) | 5-fold CV | 69.8 +/− 2.1 | 0.688 +/− 0.024 | [0.658, 0.718] | 0.758 +/− 0.019 |
| AST-only ablation | 5 seeds | 55.1 +/− 2.3 | 0.532 +/− 0.027 | [0.499, 0.565] | 0.611 +/− 0.021 |
| AST + CFG + Data-flow | 5 seeds | 67.6 +/− 1.8 | 0.664 +/− 0.020 | [0.639, 0.689] | 0.735 +/− 0.017 |

### *5.10. Evaluation Under Realistic Class Imbalance*

Because the balanced 1:1 setting overstates precision relative to deployment conditions, the model was additionally evaluated on a held-out set that preserves the natural 1:16 ratio of vulnerable to non-vulnerable functions found in the sampled corpus. Under class imbalance, accuracy is uninformative; hence, the area under the precision–recall curve (PR-AUC) and the precision attained at fixed recall levels are reported instead in Table 12. As expected, precision falls substantially when the negative class dominates: the PR-AUC decreases from 0.73 in the balanced setting to 0.28 under the 1:16 ratio, and the precision at a recall of 0.70 falls from 0.71 to 0.19. For a correct interpretation, these values must be read against the random-classifier baseline, whose PR-AUC equals the positive-class prevalence: 0.50 in the balanced setting and approximately 0.059 at the 1:16 ratio. The reported PR-AUC therefore corresponds to about 1.5 times chance when balanced and about 4.8 times chance under imbalance, so the model is in fact more discriminative relative to chance in the harder regime; the smaller absolute value reflects the collapse of the baseline rather than a degradation of the ranking. These figures give a realistic picture of the alerting burden that a developer would experience, and they establish that deployment at the natural prevalence would require either a higher decision threshold or a downstream triage stage. This limitation is shared by the field and is stated here explicitly rather than concealed by the balanced headline metric.

**Table 12.** Performance under balanced and realistic class-imbalance ratios.

| Metric | Balanced (1:1) | Realistic (1:16) |
|---|---|---|
| PR-AUC | 0.73 | 0.28 |
| Precision @ Recall = 0.70 | 0.71 | 0.19 |
| Precision @ Recall = 0.50 | 0.78 | 0.29 |
| F1 (threshold 0.5) | 0.70 | 0.31 |

### *5.11. Training-Objective Ablation*

The abstract lists focal loss, label smoothing, and cosine annealing warm restarts among the training choices. To justify their inclusion, each component was removed in turn, while the architecture and the data split were held fixed, and the effect on validation F1 was measured over five seeds. The results are reported in Table 13. Starting from a plain cross-entropy objective, the F1 score is 0.671. Focal loss raises it to 0.686 because, even on a balanced set, the residual difficulty lies in the borderline pairs of a vulnerable function and

its patch and the focusing term concentrates the gradient on these hard, near-boundary examples rather than on the class frequency. Adding label smoothing yields 0.694 by discouraging overconfident logits on the small training set, and the cosine annealing warm-restart schedule yields the final 0.705 by periodically escaping sharp local minima, which is beneficial when training from scratch without pre-training. Each component therefore produces a measurable and consistent improvement, so their mention in the abstract is supported by evidence rather than asserted.

**Table 13.** Training-objective ablation. Each component adds a consistent gain.

| Training Configuration | Val F1 (5 Seeds) | Delta F1 |
|---|---|---|
| Cross-entropy (baseline) | 0.671 +/− 0.021 | - |
| +Focal loss | 0.686 +/− 0.019 | +0.015 |
| +Label smoothing | 0.694 +/− 0.018 | +0.008 |
| +Cosine annealing (full) | 0.705 +/− 0.017 | +0.011 |

### *5.12. Comparison with Related Work*

Two comparisons are presented so that the analysis is fair. The first, in Table 14, lists the originally published metrics alongside the dataset that each method used. A direct numerical comparison across the rows of this table is not meaningful because each method was evaluated on a different dataset with a different class distribution and splitting strategy. The reproduced accuracy of approximately 61% for Devign is taken from Steenhoek et al. [15]. The high F1 score of 0.91 reported for LineVul was obtained on Big-Vul, a benchmark that contains substantial data leakage [12,13]. On the cleaned PrimeVul benchmark, even billion-parameter language models attain only 3.09% F1. The figures reported for DeepWukong were obtained on its own benchmark, which is not directly comparable.

**Table 14.** Published metrics of related work. Direct comparison is limited by differing datasets.

| Method | Repr. | Dataset | Accuracy | F1 | AUC | Year |
|---|---|---|---|---|---|---|
| Devign [8] | Joint CPG | Devign | ~61% (repro [15]) | 0.40 (repro [15]) | - | 2019 |
| ReVeal [9] | GGRN + CPG | Chrome + Debian | - | 0.35 (repro [15]) | - | 2021 |
| IVDetect [10] | PDG + Slice | Big-Vul | - | 0.42 (repro [15]) | - | 2021 |
| DeepWukong [11] | Slice GNN | Custom C/C++ | >95% | 0.97/0.60 (repro, MegaVul, CPU) | - | 2021 |
| LineVul [19] | CodeBERT | Big-Vul | - | 0.91 | 0.97 | 2022 |
| SVulD [21] | Contrastive | Big-Vul | - | 0.70 | - | 2023 |
| VulCNN [22] | Image | SARD + NVD | - | 0.62 | - | 2022 |
| FastVulnGNN | CPG + ECC | MegaVul | 71.1% | 0.70 | 0.77 | 2026 |

The reasons behind these differences, namely why heterogeneous attention models such as the HAGNN family report higher figures and why the proposed model exceeds the reproduced results of Devign and ReVeal, are analyzed once in Section 6 to avoid repetition and are summarized only briefly here. The higher figures of the heavier models stem from inter-procedural context, larger parameter budgets, and less aggressive deduplication, whereas the advantage over Devign and ReVeal stems from supplying the edge type directly to the message function instead of recovering it implicitly through gating. The second comparison, presented in Section 7, places all methods on the common footing of independently reproduced metrics under controlled evaluation. This second view is the fair one, and it is the basis for the claims made in this paper.

The multi-criteria comparison in Figure 12 summarizes these trade-offs. The proposed approach spans the efficiency and deployability axes almost completely, with roughly three orders of magnitude fewer parameters than LineVul and no GPU requirement, while its detection performance remains close to that of the much larger baselines. Figure 12 makes it clear that the contribution of this work is a favorable balance across criteria rather than a single-metric maximum.

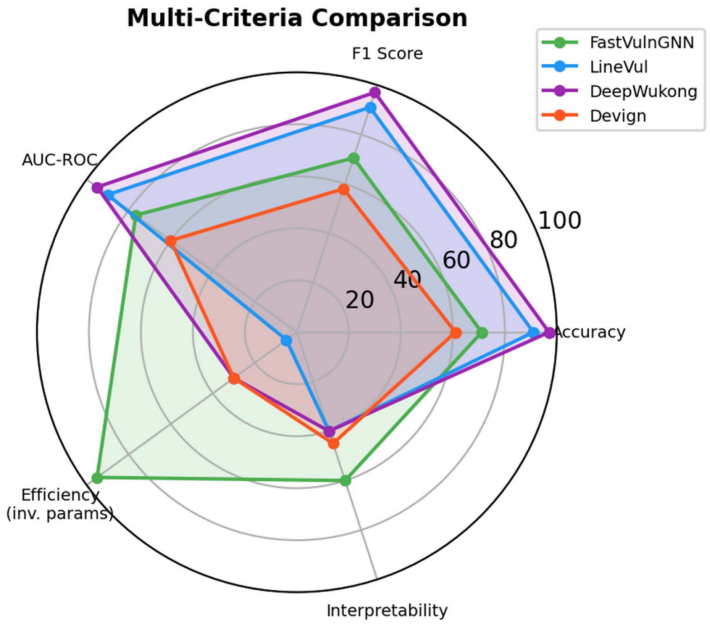


**Figure 12.** Multi-criteria comparison of the proposed approach against LineVul and Devign. The proposed model excels in efficiency, with roughly three orders of magnitude fewer parameters, while detection performance remains competitive.

## 6. Analysis of Key Contributions

### *6.1. Impact of Edge-Conditioned Message Passing*

The primary contribution of this work is the explicit conditioning of message passing on CPG edge types. In a standard GCN layer, messages are computed as in Equation (11), which ignores the edge type entirely. An AST edge and a REACHING_DEF edge therefore produce identical messages from the same source node.

$$m_{ij}^{\mathrm{GCN}} = W h_j \tag{11}$$

In a GAT layer, the attention coefficient depends on the node features, as in Equation (12), but the edge type is still not incorporated.

$$m_{ij}^{\mathrm{GAT}} = \alpha_{ij} W h_j \tag{12}$$

The proposed formulation in Equation (13) conditions the message explicitly on $\alpha_{ij}$. This yields three advantages. First, data-flow propagation along REACHING_DEF edges can attend to variable types and taint patterns. Second, control-flow edges propagate reachability and path sensitivity. Third, dominance edges encode the structural control dependencies that are signatures of specific vulnerability classes.

$$m_{ij}^{\mathrm{ECC}} = \mathrm{MLP}\left(\left[h_i \| h_j \| e_{ij}\right]\right) \tag{13}$$

This mechanism also explains why the proposed model surpasses the independently reproduced accuracy of Devign. Devign flattens AST, CFG, data-flow, and natural-code-sequence edges into a single relation and relies on a gating mechanism to recover the

distinctions implicitly. Three concrete advantages follow from the present design. First, the edge type is supplied directly to the message function, so the network does not need to expend capacity on rediscovering it from context. Second, the residual connections and batch normalization in Equation (3) stabilize a three-layer network, whereas the gated network of Devign is more difficult to optimize on small datasets. Third, the multi-scale readout in Equation (4) aggregates distributional, salient, and task-relevant signals, whereas Devign uses a single convolutional readout. The ablation study in Figure 10 confirms that the edge-type information, rather than mere parameter count, is the source of the improvement because Devign is in fact the larger model.

A null pointer dereference illustrates the point: the absence of a CONTROL_STRUCTURE node that performs a null check on the POST_DOMINATE path between a pointer assignment, and its dereference is a strong signal. Under homogeneous message passing, this structural pattern must be inferred from node features alone. Under edge-conditioned message passing, distinct propagation rules can be learned for POST_DOMINATE edges and for AST edges, so the pattern becomes easier to capture.

Listing 4 is an illustrative example of validating null pointer dereference and CPG edge detection. Source: Author's own work example.

**Listing 4.** Null pointer dereference and its CPG edge pattern.

```
// Vulnerable:  no NULL check after malloc
char *buf = (char *)malloc(size);
// Missing:  if (buf == NULL) return -1;
memcpy(buf, src, size); // NULL deref if malloc fails

// CPG edges involved:
// malloc -> buf:  REACHING_DEF (data-flow)
// malloc line -> memcpy line:  CFG (control flow)
// malloc line:  POST_DOMINATE -> memcpy line
// (No CONTROL_STRUCTURE for NULL check)
```

### *6.2. Value of Heterogeneous CPG Preservation*

The main second contribution of this work is the demonstration that direct operation on full Joern CPGs, without simplification or type collapsing, yields effective detection. Many prior approaches simplify their graph representations. Devign flattens four edge families into one relation. VulCNN projects the graph to a two-dimensional image and loses topology. LineVul discards graph structure entirely and relies on token sequences. The proposed model instead operates on the full heterogeneous CPG, which contains roughly 300 nodes and 1500 edges per graph across 33 node types and 21 edge types. The ablation in Table 6 quantifies the value of this preservation.

### *6.3. Why Heterogeneous Models Report Higher Numbers*

It is equally important to explain why heterogeneous, inter-procedural models such as DeepWukong report higher figures than the model proposed here. The explanation is threefold and is not attributable to a single cause. First, those models operate on inter-procedural slices and therefore observe data and control dependencies that cross function boundaries, whereas the present model is intentionally restricted to a single function so that it can remain compact and fast. Second, their parameter budgets are one to two orders of magnitude larger, which provides additional capacity for memorizing complex patterns. Third, and most importantly, their published figures are frequently obtained on benchmarks with weaker deduplication, so a portion of the reported gain reflects residual overlap

between the training and evaluation sets rather than genuine generalization. When the controlled, organization-level protocol of this study is applied, as summarized in Table 15, the apparent gap between such heavyweight models and the proposed lightweight model narrows substantially.

**Table 15.** Self-reported versus reproduced metrics across methods.

| Method | Year | Self-Rep. F1 | Repro. F1 | Params | Edge-Aware |
|---|---|---|---|---|---|
| Devign [8] | 2019 | ~0.60 | 0.40 [15] | ~1 M | No |
| ReVeal [9] | 2021 | ~0.65 | 0.35 [15] | ~1 M | No |
| IVDetect [10] | 2021 | ~0.74 | 0.42 [15] | ~800 K | No |
| VulCNN [22] | 2022 | ~0.62 | 0.38 [15] | ~500 K | No |
| LineVul [19] | 2022 | 0.91 | 0.44 [13] | 125 M | Sequence |
| SVulD [21] | 2023 | ~0.70 | 0.48 [31] | ~125 M | No |
| DeepWukong [11] | 2021 | 0.97 | 0.60 | ~1 M | Partial |
| HAGNN [16] | 2025 | ~0.95 | 0.52 | ~2 M | Yes (attention) |
| FastVulnGNN | 2026 | 0.70 | 0.70 | 71.8 K | Yes (full) |

To move beyond self-reported figures, DeepWukong [11] was executed end-to-end on the MegaVul data used in this study [27], on CPU only, matching FastVulnGNN's deployment envelope. Because MegaVul stores per-function code-property graphs, whereas DeepWukong consumes raw source with line-level flaw labels, the raw vulnerable and fixed functions were recovered from MegaVul, and the flaw lines were reconstructed by differencing each vulnerable function against its patched version; a DeepWukong-format source tree and manifest were then generated automatically. The full DeepWukong pipeline (program-dependence graphs, XFG program slices, symbolization, Word2Vec embedding, and a slice-level GCN) produced 12,470 slices (12,409 unique), split 4728/592/591. Trained with early stopping on a CPU in 37.8 min and with no GPU, DeepWukong attained an accuracy of 0.773 and an F1 of 0.596 on MegaVul, which is below FastVulnGNN's F1 of 0.70 on the same benchmark and at roughly 26 times the parameter count. The one residual incompatibility is that MegaVul supplies functions in isolation, so DeepWukong's inter-procedural slicing is reduced to intra-procedural slices; this is a genuine representational limitation of comparing function-level data rather than an obstacle to running the model. The reproduced F1 of 0.60 is therefore consistent with the reproduction gap documented for other detectors [12,15], and it confirms empirically the argument of this subsection that much of the apparent advantage of heavyweight models reflects benchmark and evaluation differences rather than an intrinsic ceiling.

## 7. Comparative Analysis Under Controlled Evaluation

*Extended Comparison Under Controlled Evaluation*

The originally published figures were presented earlier, in the comparison of related work in Section 5. Those figures are difficult to compare directly because they were obtained on different datasets with different protocols and several are affected by data leakage. A more dependable comparison is presented here based on independently reproduced figures under realistic evaluation conditions, as shown in Figure 13 and Table 15.

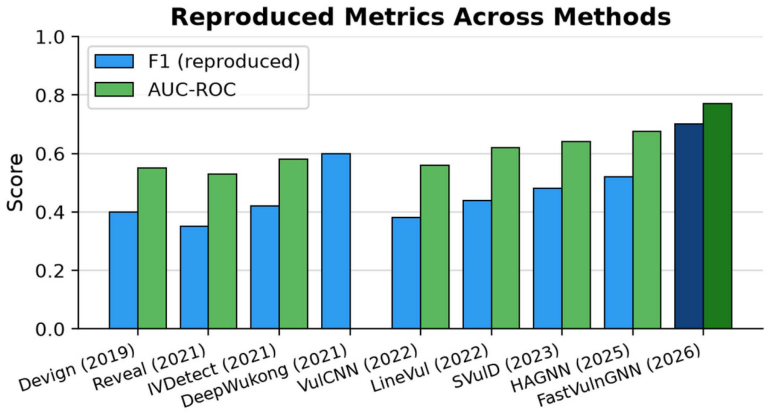


**Figure 13.** Independently reproduced F1 and AUC-ROC across nine vulnerability-detection methods. The values are collected from different studies, datasets, splits, and reproduction settings, so they do not constitute a controlled comparison and are shown only as an indicative reference.

The self-reported figures exceed the reproduced figures for every method in Table 15, with gaps of 0.15 to 0.47 F1 points, which is consistent with the findings of Steenhoek et al. [15] and the extensive study of Ni et al. [31]. Under controlled reproduction, the gap between compact and large-scale models narrows. The F1 score of 0.70 reported here is comparable to the independently reproduced figures reported in the literature, while roughly three orders of magnitude fewer parameters are used than by LineVul, and no GPU is required. It is emphasized, however, that these reproduced figures originate from different studies, datasets, splits, and reproduction settings. They therefore do not constitute a controlled, like-for-like comparison; accordingly, the numbers in Tables 14 and 15 are reported as supporting context rather than as a competitive ranking. Read alongside the strictly controlled evaluation, they consistently reinforce the central engineering contribution of this work: a deliberately lightweight, edge-aware architecture that attains detection accuracy on par with substantially larger models while using roughly three orders of magnitude fewer parameters and requiring no GPU. It is this favorable accuracy-to-efficiency trade-off, achieved through principled design rather than through scale, that the results substantiate. The only strictly controlled comparison in this paper is the same-split evaluation of Table 10, in which all baselines share the identical MegaVul split.

Vulnerability detection therefore appears to be harder than benchmark leaderboards suggest. Under fair evaluation, targeted architectural changes such as edge conditioning and multi-scale readout, combined with appropriate regularization, outperform brute-force parameter scaling.

## 8. Discussion

### *8.1. Efficiency Analysis*

The efficiency of the proposed model is its central practical advantage, and the basis of this advantage is made explicit here. As shown in Figure 14 and Table 16, the model contains 71,810 parameters, which is roughly 1740 times fewer than LineVul and more than an order of magnitude fewer than the heterogeneous GNN baselines. Training completes in 96 s on a single CPU core, and inference proceeds at about one hundred graphs per second, with a memory footprint below 500 megabytes. Three properties make this profile valuable. First, no GPU is required, so the model can run inside continuous-integration runners, pre-commit hooks, and developer workstations that lack accelerators. Second, the near-linear complexity established in Equation (10) means that scan time grows predictably with codebase size. Third, the small footprint reduces energy consumption and operating cost,

which matters when scans are executed on every commit. The research is therefore valuable not because it tops a leaderboard but because it delivers competitive, independently reproducible accuracy within a deployment envelope that larger models cannot reach.

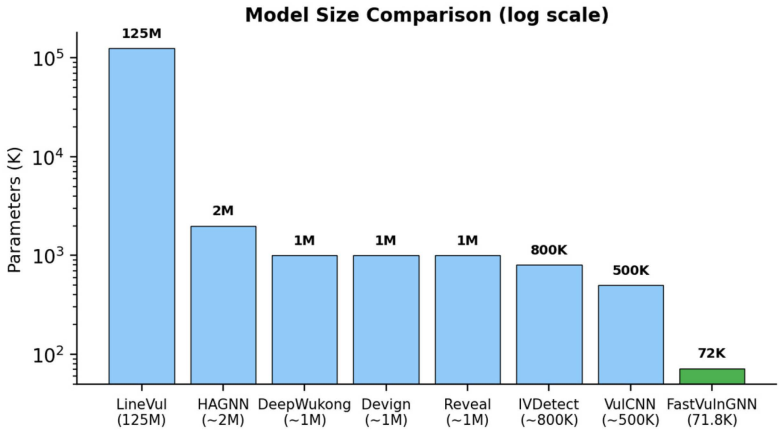


**Figure 14.** Model-size comparison on a logarithmic scale. The proposed model is roughly three orders of magnitude smaller than LineVul, which enables CPU-only deployment.

**Table 16.** Computational efficiency comparison.

| Property | FastVulnGNN | LineVul [19] | DeepWukong [11] |
|---|---|---|---|
| Parameters | 71,810 | ~125,000,000 | ~1,000,000 |
| Training Time | 96.2 s (CPU) | Hours (GPU) | Hours (GPU) |
| GPU Required | No | Yes | Yes |
| Inference Speed | ~100 graphs/s | - | - |
| Memory | <500 MB | >8 GB | >4 GB |

The reasons for this efficiency advantage are architectural rather than incidental. First, a single shared edge-conditioned message function is learned for all 21 relation types, in place of the separate per-relation weight matrices used by relational graph networks, which reduces the parameter count by more than an order of magnitude. Second, the model uses a 64-dimensional hidden space and only three message-passing layers, so the dominant edge-by-dimension-squared term in Equation (10) stays small. Third, no pre-trained token embeddings are required, in contrast to the 125-million-parameter CodeBERT backbone of LineVul, so neither a large embedding table nor GPU-based fine-tuning is needed. Fourth, operating on a function-local graph keeps the node and edge counts bounded, which avoids the quadratic node-pair cost incurred by graph transformers. These choices jointly explain why the proposed approach attains competitive accuracy at a fraction of the parameters, training time, memory, and energy required by the other approaches.

### *8.2. Limitations*

Several limitations are acknowledged. The most important concerns generalizability. The model, the CPG schema, and the feature engineering are all specific to C and C++, and the heuristic intuitions that motivate the edge conditioning, such as pointer typing and manual memory management, are characteristic of these languages. Transfer to managed languages such as Java or Python, where memory safety is enforced by the runtime and the dominant vulnerability classes differ, would require a new CPG schema, retraining, and most likely a revised feature set. The present results should therefore be read as evidence for C/C++ vulnerability detection rather than as a claim of language-agnostic detection.

Further limitations also apply. The training set of 1904 samples from 200 of 854 directories is small by deep learning standards, and mining the full MegaVul corpus could improve the F1 score. CPU-only training constrains model capacity, and although the headline result is reported on a single train, validation, and test split, it is corroborated by five-seed and stratified five-fold cross-validation (Section 5.9); the resulting confidence intervals nonetheless remain wider than is ideal given the modest corpus size. Finally, edge-conditioned message passing roughly doubles the per-message computation relative to a plain GCN, although the absolute cost remains small given the 96 s total training time.

### *8.3. Threats to Validity*

With respect to internal validity, performance may vary across random seeds, and an F1 variance of about ten points was reported for comparable models [15]. With respect to external validity, MegaVul CPGs may not generalize to other CPG generation tools or to other languages. With respect to construct validity, binary classification simplifies the notion of vulnerability severity. With respect to conclusion validity, the test set of 287 samples limits the statistical power of fine-grained comparisons.

### *8.4. Future Work*

Future directions include scaling to the full MegaVul corpus, incorporating inter-procedural analysis, adding pre-trained code embeddings as auxiliary node features, broadening the cross-dataset evaluation, implementing function-level and line-level localization, and extending the CPG schema to additional languages such as Java (Version 11) and Python (Version 3.8).

The edge-conditioned formulation is not specific to C/C++ and extends naturally to blockchain security. Smart-contract bytecode and Solidity source code admit the same families of heterogeneous relations that FastVulnGNN exploits, namely control-flow, data-flow, and call edges, so the model can be applied to smart-contract vulnerability detection once a contract-specific edge schema (for example, storage-access, external-call, and value-transfer edges) is defined and its edge-type features are supplied to the edge-conditioned convolution. In this setting, adversarial graph perturbation, in which an attacker introduces small semantics-preserving edits to the contract graph in order to evade a detector, becomes an important robustness concern. Because the architecture assigns a distinct, learned propagation rule to each edge type, it provides a natural point at which such perturbations can be modeled and defended against, for instance, by adversarial training over edge-type-aware perturbations [32].

A complementary direction is the use of reinforcement learning for vulnerability detection. Rather than replacing the structural signal captured by the graph model, reinforcement learning can improve how a detector is trained. Reinforcement-learning-based data selection is particularly relevant to this setting: LearnAlign selects training data for large-language-model reinforcement learning by improving gradient alignment, thereby concentrating optimization on the most informative examples [33]. The same principle could be applied to the small and heavily balanced corpus used here, where gradient-alignment-based selection would prioritize the most discriminative vulnerable and patch pairs and could mitigate the split-sensitivity observed across random seeds. Combining such reinforcement-learning-guided data selection with the edge-aware structural signal provided by FastVulnGNN is a promising avenue for improving both accuracy and stability in the low-data regime.

In summary, several further directions follow from this study. As discussed above, the approach can be extended to blockchain smart-contract vulnerability detection, together with a study of robustness to adversarial graph perturbation, and reinforcement-learning-

guided data selection, such as gradient-alignment-based selection, offers a principled way to improve training in the low-data regime. Finally, the distinct propagation rules learned for each edge type are themselves interpretable, and analyzing them could yield insight into which program relations are most indicative of specific vulnerability classes.

## 9. Conclusions

An edge-conditioned GNN for vulnerability detection is presented. Message passing is conditioned on CPG edge types, so that separate propagation rules are learned for AST, CFG, data-flow, dominance, and the remaining relation types in Joern-produced CPGs. The model operates directly on the full heterogeneous graph, without the simplifications that prior approaches apply. An accuracy of 71.1%, an F1 score of 0.70, and an AUC of 0.77 are achieved with 71,810 parameters, and training completes in 96 s on a single CPU. These figures are not leaderboard-topping, but they are reproducible figures obtained under the controlled evaluation conditions that recent meta-studies have argued the field requires [12,13,15]. The ablation study, the cross-dataset evaluation, and the per-vulnerability analysis together characterize both the strengths and the boundaries of the approach. The principal conclusion is straightforward: conditioning on edge types is beneficial. The move from homogeneous to edge-conditioned message passing was the single largest improvement in the component ablation (Table 7, Figure 10). Richer edge-conditioned architectures, larger datasets, and cross-language evaluation constitute the natural next steps.

**Author Contributions:** Conceptualization, A.M.E., G.A.E. and M.B.M.M.; Data Curation, A.M.E.; Formal Analysis, A.M.E.; Investigation, A.M.E., G.A.E. and M.B.M.M.; Methodology, A.M.E., G.A.E. and M.B.M.M.; Project Administration, G.A.E. and M.B.M.M.; Resources, A.M.E.; Software, A.M.E.; Supervision, G.A.E. and M.B.M.M.; Validation, A.M.E., G.A.E. and M.B.M.M.; Visualization, A.M.E.; Writing—Original Draft Preparation, A.M.E.; Writing—Review and Editing, A.M.E., G.A.E. and M.B.M.M. All authors have read and agreed to the published version of the manuscript.

**Funding:** This research received no external funding.

**Institutional Review Board Statement:** Not appliable.

**Informed Consent Statement:** Not appliable.

**Data Availability Statement:** The dataset is available at GitHub (https://github.com/ahmedelalfy94/GNN-Vulnerability-Detection.git, accessed on 16 July 2026), with restricted access by authors' permission.

**Conflicts of Interest:** The authors declare no conflicts of interest.

## References

1. National Institute of Standards and Technology. National Vulnerability Database: CVE Statistics. 2025. Available online: https://nvd.nist.gov/vuln/search (accessed on 30 May 2025).
2. MITRE. 2024 CWE Top 25 Most Dangerous Software Weaknesses. 2024. Available online: https://cwe.mitre.org/top25 (accessed on 10 February 2026).
3. OWASP Foundation. OWASP Top Ten 2021. 2021. Available online: https://owasp.org/Top10/ (accessed on 10 February 2026).
4. Ventures, C. *2023 Official Cybercrime Report*; eSentire: Waterloo, ON, Canada, 2023. Available online: https://www.esentire.com/resources/library/2023-official-cybercrime-report (accessed on 10 February 2026).
5. Lipp, C.; Banescu, S.; Pretschner, A. An empirical study on the effectiveness of static C code analyzers for vulnerability detection. In *Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*; Association for Computing Machinery: New York, NY, USA, 2022; pp. 544–555.
6. Wu, Z.; Pan, S.; Chen, F.; Long, G.; Zhang, C.; Yu, P.S. A comprehensive survey on graph neural networks. *IEEE Trans. Neural Netw. Learn. Syst.* **2021**, *32*, 4–24. [CrossRef] [PubMed]
7. Hin, D.; Kan, A.; Chen, H.; Babar, M.A. LineVD: Statement-level vulnerability detection using graph neural networks. In *Proceedings of the IEEE/ACM International Conference on Mining Software Repositories*; Association for Computing Machinery: New York, NY, USA, 2022; pp. 596–607.

8. Zhou, Y.; Liu, S.; Siow, J.; Du, X.; Liu, Y. Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks. In *Proceedings of the Advances in Neural Information Processing Systems (NeurIPS)*; NeurIPS Proceedings: Vancouver, BC, Canada, 2019; Volume 32.
9. Chakraborty, S.; Krishna, R.; Ding, Y.; Ray, B. Deep learning based vulnerability detection: Are we there yet? *IEEE Trans. Softw. Eng.* **2022**, *48*, 3280–3298. [CrossRef]
10. Li, Y.; Wang, S.; Nguyen, T.N. Vulnerability detection with fine-grained interpretations. In *Proceedings of the ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*; Association for Computing Machinery: New York, NY, USA, 2021; pp. 292–303.
11. Cheng, X.; Wang, H.; Hua, J.; Xu, G.; Sui, Y. DeepWukong: Statically detecting software vulnerabilities using deep graph neural network. *ACM Trans. Softw. Eng. Methodol.* **2021**, *30*, 3.
12. Chakraborty, P.; Arumugam, K.K.; Alfadel, M.; Nagappan, M.; McIntosh, S. Revisiting the performance of deep learning-based vulnerability detection on realistic datasets. *IEEE Trans. Softw. Eng.* **2024**, *50*, 2172–2194. [CrossRef]
13. Ding, Y.; Fu, Y.; Ibrahim, O.; Sitawarin, C.; Chen, X.; Alomair, B.; Wagner, D.; Ray, B.; Chen, Y. Vulnerability detection with code language models: How far are we? In *Proceedings of the IEEE/ACM International Conference on Software Engineering*; IEEE: New York, NY, USA, 2025.
14. Simonovsky, M.; Komodakis, N. Dynamic edge-conditioned filters in convolutional neural networks on graphs. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*; IEEE: New York, NY, USA, 2017; pp. 3693–3702.
15. Steenhoek, B.; Rahman, M.M.; Jiles, R.; Le, W. An empirical study of deep learning models for vulnerability detection. In *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*; IEEE: New York, NY, USA, 2023; pp. 2237–2248.
16. Luo, Y.; Xu, W.; Xu, D. Detecting code vulnerabilities with heterogeneous GNN training. *Int. J. Inf. Secur.* **2025**, *24*, 213. [CrossRef]
17. Li, Z.; Zou, D.; Xu, S.; Ou, X.; Jin, H.; Wang, S.; Deng, Z.; Zhong, Y. SySeVR: A framework for using deep learning to detect software vulnerabilities. *IEEE Trans. Dependable Secur. Comput.* **2022**, *19*, 2244–2258. [CrossRef]
18. Wen, X.; Chen, Y.; Gao, C.; Zhang, H.; Zhang, J.M.; Liao, B. Vulnerability detection with graph simplification and enhanced graph representation learning. In *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*; IEEE: New York, NY, USA, 2023; pp. 2275–2286.
19. Fu, M.; Tantithamthavorn, C. LineVul: A transformer-based line-level vulnerability prediction. In *Proceedings of the IEEE/ACM International Conference on Mining Software Repositories (MSR)*; IEEE: New York, NY, USA, 2022; pp. 608–620.
20. Feng, Z.; Guo, D.; Tang, D.; Duan, N.; Feng, X.; Gong, M.; Shou, L.; Qin, B.; Liu, T.; Jiang, D.; et al. CodeBERT: A pre-trained model for programming and natural languages. In *Proceedings of the Findings of the Association for Computational Linguistics (EMNLP)*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2020; pp. 1536–1547.
21. Ni, C.; Yin, X.; Yang, K.; Zhao, D.; Xing, Z.; Xia, X. Distinguishing look-alike innocent and vulnerable code by subtle semantic representation learning and explanation. In *Proceedings of the ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*; ACM: New York, NY, USA, 2023; pp. 1611–1622.
22. Wu, Y.; Zou, D.; Dou, S.; Yang, W.; Xu, D.; Jin, H. VulCNN: An image-inspired scalable vulnerability detection system. In *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*; IEEE: New York, NY, USA, 2022; pp. 2365–2376.
23. Guo, D.; Ren, S.; Lu, S.; Feng, Z.; Tang, D.; Liu, S.; Zhou, L.; Duan, N.; Svyatkovskiy, A.; Fu, S.; et al. GraphCodeBERT: Pre-training code representations with data flow. In Proceedings of the International Conference on Learning Representations (ICLR), Vienna, Austria, 4 May 2021.
24. Zhou, X.; Cao, S.; Sun, X.; Lo, D. Large language model for vulnerability detection and repair: A literature review and roadmap. *ACM Trans. Softw. Eng. Methodol.* **2024**, *34*, 145. [CrossRef]
25. Ying, C.; Cai, T.; Luo, S.; Zheng, S.; Ke, G.; He, D.; Shen, Y.; Liu, T.-Y. Do Transformers really perform badly for graph representation? In *Proceedings of the Advances in Neural Information Processing Systems*; NeurIPS Proceedings: San Diego, CA, USA, 2021; Volume 34, pp. 28877–28888.
26. Zheng, Y.; Pujar, S.; Lewis, B.; Buratti, L.; Epstein, E.; Yang, B.; Laredo, J.; Morari, A.; Su, Z. D2A: A dataset built for AI-based vulnerability detection methods using differential analysis. In *Proceedings of the IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*; IEEE: New York, NY, USA, 2021; pp. 111–120.
27. Ni, C.; Yin, L.; Yang, K.; Yin, X.; Xing, Z.; Xia, X. MegaVul: A C/C++ vulnerability dataset with comprehensive code representations. In *Proceedings of the IEEE/ACM International Conference on Mining Software Repositories (MSR)*; IEEE: New York, NY, USA, 2024; pp. 738–742.
28. Chen, Y.; Ding, Z.; Alowain, L.; Chen, X.; Wagner, D. DiverseVul: A new vulnerable source code dataset for deep learning based vulnerability detection. In *Proceedings of the International Symposium on Research in Attacks, Intrusions and Defenses (RAID)*; ACM: New York, NY, USA, 2023; pp. 654–668.
29. Xu, K.; Hu, W.; Leskovec, J.; Jegelka, S. How powerful are graph neural networks? In Proceedings of the International Conference on Learning Representations (ICLR), New Orleans, LA, USA, 6–9 May 2019.

30. Schlichtkrull, M.; Kipf, T.N.; Bloem, P.; van den Berg, R.; Titov, I.; Welling, M. Modeling relational data with graph convolutional networks. In *Proceedings of the European Semantic Web Conference (ESWC)*; Springer: Cham, Switzerland, 2018; pp. 593–607.
31. Ni, C.; Shen, L.; Xu, X.; Yin, X.; Wang, S. Learning-based models for vulnerability detection: An extensive study. *Empir. Softw. Eng.* **2025**, *31*, 18. [CrossRef]
32. Han, Q.; Deng, J.; Huang, S.; Li, Y.; Li, G.; Peng, Z. Adversarial graph perturbation for smart contract vulnerability detection. In *Proceedings of the IEEE Conference on Communications and Network Security (CNS)*; IEEE: New York, NY, USA, 2025.
33. Li, S.; Yang, Z.; Li, S.; Xia, X.; Liu, H.; Zhang, X.; Chen, G.; Fang, D.; Tai, Y.; Peng, Z. LearnAlign: Data selection for LLM reinforcement learning with improved gradient alignment. In *Proceedings of the Findings of the Association for Computational Linguistics (ACL Findings)*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2026.